

20 questions SQL

Jointures, GROUP BY, sous-requêtes /20



Avant ton entretien Data Analyst. Durée conseillée : 25 min · 3 paliers · corrigé détaillé.

Objectif : mesurer tes réflexes SQL réels, pas ta syntaxe par cœur. Ce test vérifie si tu vois qu'un LEFT JOIN redevient un INNER JOIN sans le vouloir, quel grain tu lis avant d'agréger, et pourquoi un NOT IN peut renvoyer zéro ligne sans planter. En entretien, c'est rarement la syntaxe qui élimine un candidat : c'est le piège qu'il n'a pas vu.

Repères : 12/20 = bases solides, 17/20 = profil qu'on retient pour la suite.

Candidat _____ Date _____ Note _____ /20

Les bases qui trient.

Sept questions sur les jointures : ce qui survit, ce qui disparaît, ce qui se duplique en silence.

Question 1. Entre INNER JOIN et LEFT JOIN de clients vers commandes, quelle affirmation est exacte ?

- a) Le LEFT JOIN conserve tous les clients; les colonnes de commandes valent NULL sans correspondance.
- b) L'INNER JOIN conserve tous les clients, même sans commande.
- c) LEFT JOIN et INNER JOIN renvoient toujours le même nombre de lignes.
- d) L'INNER JOIN remplit les colonnes manquantes avec des zéros.

Question 2. Un collègue t'envoie ce fichier export à corriger : `SELECT c.nom, o.montant FROM clients c LEFT JOIN commandes o ON c.id = o.client_id WHERE o.montant > 100.` **Que produit réellement cette requête ?**

- a) Tous les clients, avec ou sans commande, comme prévu par le LEFT JOIN.
- b) Seulement les clients ayant une commande de plus de 100 : le WHERE sur la table droite annule l'effet du LEFT JOIN.
- c) Une erreur de syntaxe, WHERE ne peut pas suivre un LEFT JOIN.
- d) Les clients sans commande, avec montant affiché à 0.

Question 3. Un analyste joint ventes à produits (une ligne produit peut apparaître plusieurs fois côté ventes) puis calcule `SUM(ventes.montant)`. Le total est plus élevé que prévu. Quelle est la cause la plus probable ?

- a) SUM ignore les valeurs négatives.
- b) Le JOIN a été fait avec RIGHT JOIN au lieu de LEFT JOIN.
- c) La clé de jointure n'est pas unique côté produits : chaque ligne de ventes est dupliquée par correspondance, et le SUM compte les doublons.
- d) Il manque un ORDER BY avant l'agrégation.

Question 4. Tu dois lister les employés qui partagent le même manager, sur une seule table `employees(id, nom, manager_id)`. Quelle approche fonctionne ?

- a) Un LEFT JOIN entre `employees` et une table `managers` séparée.
- b) Un GROUP BY `manager_id` suffit, sans jointure.
- c) Un CROSS JOIN entre `employees` et `employees`.
- d) Un SELF JOIN de `employees` avec elle-même, sur `e1.manager_id = e2.manager_id AND e1.id <> e2.id`.

Question 5. Pour la lisibilité, quelle pratique est recommandée face à un RIGHT JOIN ?

- a) Le réécrire en LEFT JOIN en inversant l'ordre des tables : le résultat est équivalent et plus facile à relire.
- b) Le remplacer systématiquement par un INNER JOIN.
- c) Ne jamais utiliser LEFT JOIN, seul RIGHT JOIN garantit l'intégrité.
- d) RIGHT JOIN et LEFT JOIN produisent des résultats différents même en inversant les tables.

Question 6. Deux tables ont une colonne `code_client` qui contient parfois NULL des deux côtés. Un JOIN ... ON `a.code_client = b.code_client` apparie-t-il les lignes où les deux valent NULL ?

- a) Oui, NULL est traité comme une valeur normale pour la comparaison.
- b) Non, NULL = NULL évalue à inconnu : deux NULL ne se joignent jamais entre eux.
- c) Oui, mais seulement en INNER JOIN.
- d) Cela dépend du nombre de lignes NULL de chaque côté.

Question 7. Tu lis cette requête écrite par un stagiaire : `SELECT * FROM commandes, clients`. Que produit-elle ?

- a) La même chose qu'un INNER JOIN classique.
- b) Une erreur, car il manque ORDER BY.
- c) Un produit cartésien : chaque ligne de commandes est associée à chaque ligne de clients, sans condition de jointure.
- d) Uniquement les lignes où les deux tables partagent un identifiant.

Le rapport qui doit tomber juste.

Sept questions sur GROUP BY : le grain change, et un chiffre juste en apparence peut partir faux dès qu'on agrège au mauvais niveau.

Question 8. Sur une table `clients` où la colonne `email` contient des NULL, quelle différence existe entre `COUNT(*)` et `COUNT(email)` ?

- a) Aucune, les deux comptent le même nombre de lignes.
- b) `COUNT(*)` ignore les NULL, pas `COUNT(email)`.
- c) `COUNT(email)` compte uniquement les doublons.
- d) `COUNT(*)` compte toutes les lignes, `COUNT(email)` ne compte que les lignes où email n'est pas NULL.

Question 9. Cette requête tourne sur un moteur en mode strict : `SELECT ville, client_id, COUNT(*) FROM clients GROUP BY ville`. Que se passe-t-il ?

- a) Erreur : `client_id` est au SELECT sans être dans le GROUP BY ni agrégé.
- b) La requête s'exécute normalement sur tous les moteurs, sans exception.
- c) `client_id` est automatiquement remplacé par NULL.
- d) Le moteur trie les `client_id` par ordre alphabétique.

Question 10. Ton manager veut la liste des villes ayant plus de 50 clients actifs. Tu dois filtrer sur `statut = 'actif'` avant de compter, puis ne garder que les villes au-dessus de 50. Où placer chaque filtre ?

- a) Les deux filtres dans WHERE.
- b) `statut = 'actif'` dans WHERE, `COUNT(*) > 50` dans HAVING.
- c) Les deux filtres dans HAVING.
- d) `statut = 'actif'` dans HAVING, `COUNT(*) > 50` dans WHERE.

Question 11. Une colonne `note` contient des valeurs et des NULL (notes non renseignées). Pour calculer la moyenne réelle des notes renseignées, quelle affirmation est correcte ?

- a) `AVG(note)` traite les NULL comme des 0, ce qui fausse la moyenne vers le bas.
- b) Il faut d'abord remplacer les NULL par 0 avec `COALESCE` pour un résultat correct.
- c) `AVG(note)` ignore nativement les NULL et ne divise que par le nombre de valeurs renseignées.
- d) `AVG(note)` renvoie une erreur si la colonne contient au moins un NULL.

Question 12. Tu lis `SELECT region, produit, SUM(quantite) FROM ventes GROUP BY region, produit`. Quel est le grain d'une ligne du résultat ?

- a) Une ligne par vente individuelle.
- b) Une ligne par région, toutes régions confondues.
- c) Une ligne par produit, toutes régions confondues.
- d) Une ligne par combinaison unique région/produit.

Question 13. Sans fonction d'agrégation, quelle requête produit le même résultat que `SELECT DISTINCT ville FROM clients` ?

- a) `SELECT ville FROM clients GROUP BY ville.`
- b) `SELECT ville FROM clients HAVING ville IS NOT NULL.`
- c) `SELECT ville FROM clients ORDER BY ville.`
- d) `SELECT ville, COUNT(*) FROM clients.`

Question 14. En entretien, on te montre `SELECT ville, COUNT(*) AS nb FROM clients GROUP BY ville ORDER BY nb DESC`, puis on enlève le `GROUP BY` en gardant le reste. Que dire ?

- a) La requête reste valide, l'alias `nb` suffit à remplacer le `GROUP BY`.
- b) Sans `GROUP BY`, `COUNT(*)` agrège toute la table en une seule ligne : le résultat par ville n'existe plus.
- c) `ORDER BY nb` devient invalide car l'alias n'est jamais utilisable en `ORDER BY`.
- d) La requête calcule correctement un `COUNT` par ville même sans `GROUP BY`.

La dernière ligne droite.

Six questions sur les sous-requêtes : celles qui plantent, celles qui renvoient zéro ligne sans erreur, et celles qu'on peut réécrire en JOIN.

Question 15. Une sous-requête scalaire utilisée dans `SELECT (SELECT prix FROM produits WHERE categorie = 'A') AS prix_ref FROM ventes` renvoie plusieurs lignes au lieu d'une seule valeur. Que se passe-t-il à l'exécution ?

- a) SQL prend automatiquement la première ligne renvoyée.
- b) SQL prend automatiquement la valeur maximale.
- c) Une erreur runtime : une sous-requête scalaire doit renvoyer exactement une valeur.
- d) Le résultat est NULL sans erreur.

Question 16. Tu lis `SELECT c.nom FROM clients c WHERE EXISTS (SELECT 1 FROM commandes o WHERE o.client_id = c.id)`. Comment est évaluée la sous-requête ?

- a) Une seule fois, avant la requête principale.
- b) Elle n'est jamais évaluée, `EXISTS` teste seulement la syntaxe.
- c) Une seule fois par valeur distincte de `client_id` dans `commandes`.
- d) Une fois par ligne de clients : c'est une sous-requête corrélée, réévaluée pour chaque client.

Question 17. `SELECT * FROM clients WHERE id NOT IN (SELECT client_id FROM blacklist)`. La table `blacklist` contient un `client_id` à NULL. Résultat obtenu ?

- a) Aucune ligne : la présence d'un NULL rend la condition `NOT IN` indéterminée pour toutes les lignes.
- b) Tous les clients sauf ceux blacklistés, le NULL est ignoré automatiquement.
- c) Une erreur de syntaxe bloquant l'exécution.
- d) Seuls les clients dont l'id est NULL sont exclus.

Question 18. Pour vérifier qu'un client a au moins une commande, sans risquer de doublons ni le piège des NULL, quelle approche privilégier ?

- a) Un JOIN classique entre clients et commandes.
- b) Un `WHERE EXISTS (...)` sur une sous-requête corrélée.
- c) Un `NOT IN` sur la liste des `client_id` de commandes.
- d) Un `IN` sur une sous-requête qui peut contenir des NULL.

Question 19. Tu essaies d'exécuter `SELECT * FROM (SELECT client_id, SUM(montant) FROM commandes GROUP BY client_id)`, sans nommer la sous-requête. Que se passe-t-il sur la plupart des moteurs ?

- a) Ça fonctionne, un alias est optionnel pour une table dérivée.
- b) Le moteur choisit automatiquement un alias par défaut.
- c) Erreur : une table dérivée en FROM doit avoir un alias dans la plupart des dialectes SQL.
- d) La requête s'exécute mais ignore le SUM.

Question 20. Tu réécris `SELECT c.nom FROM clients c WHERE EXISTS (SELECT 1 FROM commandes o WHERE o.client_id = c.id)` avec un JOIN plutôt qu'une sous-requête corrélée. À quoi dois-tu faire attention ?

- a) Rien, JOIN et EXISTS sont interchangeables sans aucune précaution.
- b) Le JOIN sera toujours plus lent, il faut éviter cette réécriture.
- c) EXISTS ne peut jamais être remplacé par un JOIN.
- d) Si un client a plusieurs commandes, un JOIN classique dupliquera la ligne client; il faut un DISTINCT ou un GROUP BY pour retrouver l'équivalence.

Solutions et explications.

Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute.

1. **a)** Le LEFT JOIN garde toutes les lignes de gauche même sans correspondance; le piège classique est de croire qu'INNER JOIN fait pareil, alors qu'il élimine les orphelines.
2. **b)** Filtrer sur une colonne de la table jointe dans le WHERE élimine les lignes où elle vaut NULL : le LEFT JOIN se comporte alors comme un INNER JOIN, piège fréquent sur les fichiers reçus tels quels.
3. **c)** Une clé de jointure non unique multiplie les lignes (fan-out); le SUM compte alors les montants en double, piège classique des JOIN posés avant une agrégation.
4. **d)** Le SELF JOIN compare des lignes d'une même table via deux alias; oublier la condition d'inégalité (id différent de id) recrée des doublons inutiles.
5. **a)** RIGHT JOIN reste rare et moins lisible; l'inverser en LEFT JOIN en changeant l'ordre des tables donne un résultat équivalent, convention de style plutôt que règle stricte du standard.
6. **b)** NULL = NULL évalue à inconnu en logique ternaire SQL : deux NULL ne se joignent jamais, piège classique quand on espère qu'ils matchent l'un avec l'autre.
7. **c)** Sans clause ON (ou avec une virgule entre tables), SQL produit un produit cartésien : chaque ligne de gauche est associée à chaque ligne de droite, piège qui fait exploser silencieusement le nombre de lignes.
8. **d)** COUNT(*) compte toutes les lignes, COUNT(colonne) ignore les NULL; confondre les deux fausse un décompte dès qu'une colonne est partiellement renseignée.
9. **a)** En SQL strict, une colonne du SELECT hors GROUP BY et non agrégée est rejetée. Certains moteurs non stricts (MySQL en configuration ancienne) tolèrent une valeur arbitraire, mais s'y fier est un piège de portabilité.
10. **b)** WHERE filtre les lignes avant l'agrégation, HAVING filtre le résultat agrégé après; mettre un filtre sur COUNT(*) dans WHERE est une erreur classique de débutant.
11. **c)** AVG() ignore nativement les NULL et ne divise que par les valeurs renseignées; le piège est de croire qu'il les traite comme des 0, ce qui ferait chuter la moyenne à tort.
12. **d)** Un GROUP BY multi-colonnes produit une ligne par combinaison unique des colonnes citées; oublier ce grain fait mal lire un rapport agrégé.
13. **a)** GROUP BY sans fonction d'agrégation déduplique comme DISTINCT; le piège est de croire que GROUP BY sert uniquement à agréger.
14. **b)** Sans GROUP BY, COUNT(*) agrège toute la table en une seule ligne, le détail par ville disparaît. Note : l'alias en ORDER BY est toléré par la plupart des moteurs mais pas garanti partout, un détail à vérifier avant de généraliser.
15. **c)** Une sous-requête scalaire doit renvoyer exactement une valeur; en renvoyer plusieurs déclenche une erreur runtime, piège fréquent quand le filtre de la sous-requête n'est pas assez précis.
16. **d)** Une sous-requête corrélée est réévaluée pour chaque ligne de la requête externe; EXISTS s'appuie sur ce mécanisme pour tester une existence ligne par ligne.
17. **a)** NOT IN avec une sous-requête contenant un NULL renvoie inconnu pour chaque comparaison, donc aucune ligne : piège documenté sur tous les moteurs majeurs, à éviter en préférant NOT EXISTS.
18. **b)** EXISTS évite la duplication qu'un JOIN peut produire quand un client a plusieurs commandes, et échappe au piège des NULL qui casse NOT IN et IN.
19. **c)** Une table dérivée en FROM doit avoir un alias dans la plupart des dialectes (PostgreSQL, MySQL, SQL Server); l'oublier est une erreur fréquente en sous-requête imbriquée.
20. **d)** Réécrire une sous-requête corrélée en JOIN peut dupliquer la ligne externe si la relation n'est pas 1 :1; il faut alors un DISTINCT ou un GROUP BY pour retrouver l'équivalence, piège de cardinalité classique.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

